

Danmarks
Tekniske
Universitet



Python and HPC Final Report

AUTHORS

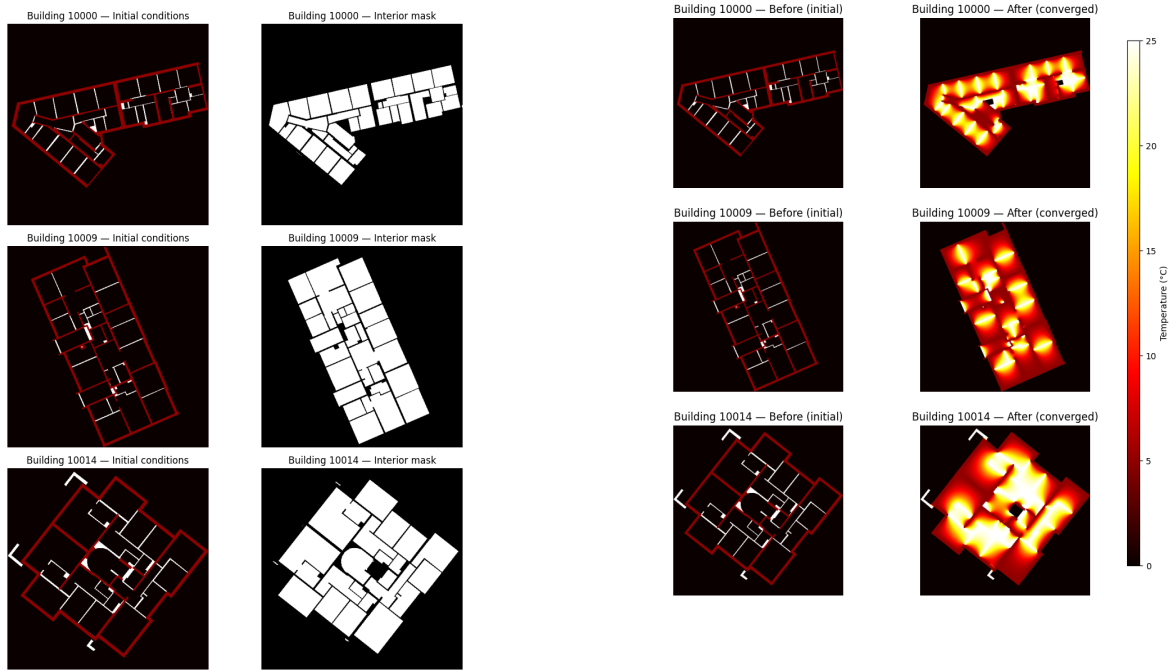
Kyle Nathan Yahya - s252786
Botond Rendes - s253343
Carl Daniel Karlsson - s253791
Emiliano Gomez Cabello- s252668

May 31, 2026

Contents

1 Exercises	1
1.1 Question 1	1
1.2 Question 2	1
1.3 Question 3	1
1.4 Question 4	2
1.5 Question 5	2
1.5.1 Part 1	2
1.5.2 Part 2	2
1.5.3 Part 3	3
1.5.4 Part 4	3
1.6 Question 6	3
1.7 Question 7	5
1.8 Question 8	6
1.8.1 Kernel structure	6
1.8.2 Helper function structure	7
1.8.3 Performance comparison	7
1.8.4 Estimated runtime for all floor plans	8
1.9 Question 9	8
1.9.1 Part 1	8
1.9.2 Part 2	9
1.9.3 Part 3	9
1.10 Question 10	9
1.11 Question 11	10
1.12 Question 12	11
List of Figures	I
List of Tables	II

1 Exercises



(a) Description of left image.

(b) Description of right image.

Figure 1: A comparison between two spanning subfigures.

1.1 Question 1

Figure 1a shows visual representations of the initial temperatures and the interior mask of certain floors.

1.2 Question 2

A simple simulation of a subset of 20 different floors was done in 233.6 seconds. If we assume that each floor takes about 11.7 seconds, it is estimated to take 889.8 minutes to run a complete simulation of the 4571 floors.

1.3 Question 3

Figure 1b shows visual representations of the initial temperatures and the final temperature after simulation of certain floors.

1.4 Question 4

Table 1: Performance profile summary of the Jacobi solver.

Function	ncalls	tottime	percall (tot)	cumtime	percall (cum)
<code>jacobi</code>	1	5.861	5.861	5.938	5.938
<code>max</code>	3602	0.006	0.000	0.077	0.000
<code>_amax</code>	3602	0.003	0.000	0.071	0.000
<code>reduce</code>	3602	0.068	0.000	0.068	0.000
<code>copy</code>	1	0.000	0.000	0.000	0.000
<code>numpy.array</code>	1	0.000	0.000	0.000	0.000
<code>disable</code>	1	0.000	0.000	0.000	0.000
<code>_copy_dispatcher</code>	1	0.000	0.000	0.000	0.000

As observed in Table Table 1, the solver required 3602 iterations to converge, explained by the `ncalls` metric for the `max` function. The `jacobi` function accounts for approximately 98.7% of the total cumulative runtime (5.861s out of 5.938s).

Even with NumPy vectorization, the primary bottleneck is the iterative Python loop and its repeated array operations. The convergence check (`max`, `_amax`, and `reduce`) accounts for less than 2% of the runtime, meaning any optimizations there would have almost no effect on total execution time.

1.5 Question 5

An experiment was conducted using $N = 50$ buildings across 1, 2, 4, and 8 workers using static scheduling using a batch job.

1.5.1 Part 1

The system shows sub-linear scaling, as shown in Table 2. While doubling workers from 1 to 2 initially gives a near-perfect $1.93\times$ boost, adding more cores becomes less efficient due to file I/O and task distribution overhead.

1.5.2 Part 2

The parallel fraction p represents the portion of the code that scales with n processors. Following Amdahl's Law, the speedup $S(n)$ is defined as:

$$S(n) = \frac{1}{(1 - p) + \frac{p}{n}} \quad (1)$$

The estimated parallel fraction for this implementation is $p \approx 0.851$. This indicates that roughly 85.1% of the total execution time is successfully parallelized, while the remaining 14.9% remains serial.

Table 2: Measured Runtime and Speed-up ($N = 50$)

Workers	Time (s)	Speed-up
1	663.04	1.000
2	343.77	1.929
4	303.20	2.187
8	169.46	3.913

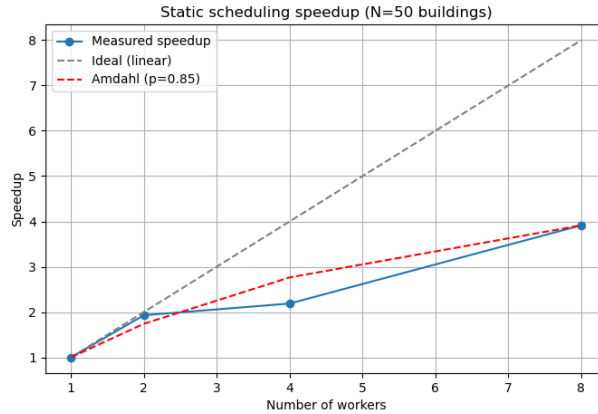


Figure 2: Speedup comparison: Measured vs. Amdahl's Law.

1.5.3 Part 3

The theoretical maximum speed-up is defined by the limit of Amdahl's Law as the number of processors $n \rightarrow \infty$:

$$\text{Max Speedup} = \frac{1}{1-p} = \frac{1}{1-0.851} \approx 6.70 \times \quad (2)$$

With an achieved speed-up of $3.91 \times$ using 8 cores, the implementation reached approximately 58.3% of its theoretical maximum performance. As seen in Figure 2.

1.5.4 Part 4

Based on the measured runtime of 169.46s for 50 buildings, the average processing time is approximately 3.39s per building. At this rate, processing the entire 4,571-building dataset is estimated to take 15,491s (258.2 minutes or 4.3 hours).

1.6 Question 6

The previous multiprocessing solution used static scheduling, where the floorplans were split into fixed chunks before the workers started. Since different floorplans need different numbers of Jacobi iterations before convergence, this can cause load imbalance. Some workers may finish early while others are still processing harder floorplans. To improve this, the parallelization was changed to dynamic scheduling using `multiprocessing.Pool.imap_unordered`, so workers receive new floorplans as soon as they finish their current one.

The experiment was run on the first 50 floorplans, matching the subset used for the static multiprocessing baseline in Question 5. Table 3 compares the static and dynamic scheduling results from our own rerun. These values are used for the main comparison because the static and dynamic measurements were obtained in the same script, job setup, and timing framework. The static rerun also agrees well with the Question 5 timings, for

example, the 8-worker static runtime was 169.60 seconds here compared with 169.46 seconds in Question 5. The largest absolute improvement was seen for 4 workers, where the runtime decreased from 287.55 seconds to 198.83 seconds.

Table 3: Comparison between static and dynamic scheduling for the NumPy CPU implementation.

Workers	Static (s)	Dynamic (s)	Improvement (s)	Speed-up vs. 1 worker
1	619.78	603.19	16.59	-
2	355.09	331.38	23.71	1.82
4	287.55	198.83	88.72	3.03
8	169.60	112.61	56.99	5.36

The speed-up also improved compared with the static version, since the dynamic version consistently had lower wall-clock time for the same number of workers. This supports the expectation that the work per floorplan is uneven, and that dynamic scheduling reduces worker idle time. In Figure 3, the ideal line corresponds to perfect linear scaling, where doubling the number of workers would exactly double the speed-up.

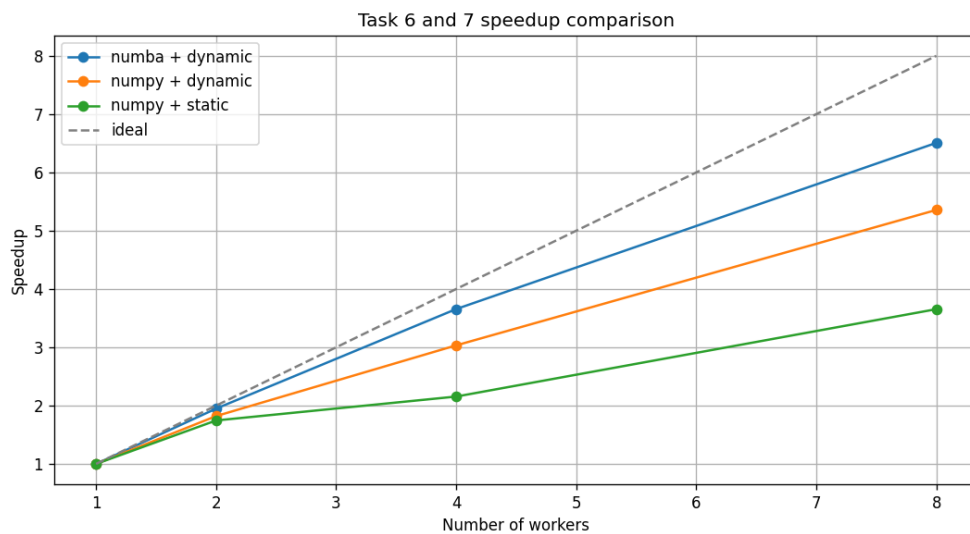


Figure 3: Measured speed-up for the CPU implementations on a subset of 50 floorplans.

The difference between `numpy + static` and `numpy + dynamic` shows the effect of dynamic scheduling.

We also tested different chunk sizes for the dynamic scheduler using the fastest CPU configuration. The runtimes were 20.85, 25.70, 29.12, and 40.34 seconds for chunk sizes 1, 2, 4, and 8, respectively. The best result was obtained with chunk size 1, which indicates that fine-grained scheduling helped balance the uneven floorplan runtimes.

1.7 Question 7

The Jacobi solver was rewritten using Numba JIT on the CPU. The original NumPy implementation used array slicing and boolean masking inside every iteration, which is concise but creates temporary arrays and repeated indexing overhead. The Numba implementation instead uses `@njit(cache=True)` and explicit nested loops over the grid. Two arrays, `u_old` and `u_new`, are used because Jacobi iteration computes every new value from the previous full grid state.

The loop order was chosen to follow NumPy's row-major memory layout:

Listing 1: Main row-major update loop used in the Numba Jacobi implementation.

```

1 for i in range(1, n_rows - 1):
2     for j in range(1, n_cols - 1):
3         if interior_mask[i - 1, j - 1]:
4             updated = 0.25 * (u_old[i, j - 1]
5                             + u_old[i, j + 1]
6                             + u_old[i - 1, j]
7                             + u_old[i + 1, j]
8                             )
9             diff = abs(u_old[i, j] - updated)
10            if diff > delta:
11                delta = diff
12            u_new[i, j] = updated
13        else:
14            u_new[i, j] = u_old[i, j]
```

This means that the inner loop moves through consecutive entries in memory, improving spatial locality and making the access pattern more cache-friendly. The interior mask is checked directly inside the compiled loop instead of using repeated masked indexing, which avoids many temporary arrays.

To avoid including Numba compilation overhead in the timing, the JIT function was warmed up once before measuring the actual runtime. On a representative floorplan, the NumPy solver took 6.19 seconds, while the Numba version took 1.31 seconds. Both versions converged after 3602 iterations and produced the same mean temperature and standard deviation, so the numerical result was preserved. This gives a speed-up of

$$\frac{6.19}{1.31} \approx 4.73.$$

The fastest measured CPU configuration combined the Numba solver with dynamic scheduling, 8 workers, and chunk size 1. This processed 50 floorplans in 21.02 seconds. Extrapolating to all 4571 floorplans gives

$$21.02 \times \frac{4571}{50} \approx 1922 \text{ s} \approx 32 \text{ min.}$$

Thus, the estimated runtime for all floorplans using this CPU solution is around 32

minutes. Compared with the 8-worker static NumPy baseline of 169.60 seconds for the 50-floorplan benchmark, the combined `numba + dynamic` solution took 21.02 seconds, corresponding to an overall speed-up of approximately 8.07x. This combined speed-up includes both the dynamic scheduling from Question 6 and the Numba optimization from Question 7.

1.8 Question 8

A custom CUDA kernel was implemented using Numba to accelerate the Jacobi solver on the GPU. Unlike the CPU implementations, where thread synchronization between iterations is handled implicitly by the operating system and CPU architecture, GPU threads within a kernel cannot synchronize across the entire grid. To work around this, the kernel was designed to perform only a single Jacobi iteration per launch. Synchronization between iterations is then achieved by returning to the host (CPU) between each kernel call, which acts as a global barrier. This means the helper function drives the iteration loop from Python, calling the kernel once per step for a fixed number of iterations, with no early stopping.

1.8.1 Kernel structure

The kernel maps one GPU thread to one grid point (i, j) . Each thread independently checks its position and updates its value:

- If the point is on the boundary of the grid, its value is copied unchanged.
- If the point is an interior cell inside the floor plan (mask is true), it is replaced by the average of its four neighbours.
- If the point is an interior cell that is a wall or fixed cell (mask is false), it is copied unchanged.

Threads are organised into 16×16 blocks, and the grid is sized to cover the full temperature array. The kernel signature takes three 2D device arrays: the current temperature field `u`, the output field `u_new`, and the interior mask.

Listing 2: CUDA kernel performing a single Jacobi iteration. Each thread handles one grid point.

```

1 def jacobi_kernel(u, u_new, mask):
2     i, j = cuda.grid(2)
3     n, m = u.shape
4     if i < n and j < m:
5         if i == 0 or i == n - 1 or j == 0 or j == m - 1:
6             u_new[i, j] = u[i, j]
7         else:
8             if mask[i - 1, j - 1]:
9                 u_new[i, j] = 0.25 * (u[i, j - 1] + u[i, j + 1])

```



```

10         + u[i - 1, j] + u[i + 1, j])
11     else:
12         u_new[i, j] = u[i, j]

```

1.8.2 Helper function structure

The helper function `jacobi_numba(u, interior_mask, max_iter)` matches the reference interface, with `atol` omitted since early stopping is not used. It performs the following steps:

1. Copies `u` and `interior_mask` to GPU device memory once before the loop.
2. Allocates a second device array `u_new` of the same shape as a write buffer.
3. Runs a Python loop for `max_iter` iterations. Each step launches the kernel and then swaps the roles of `u_dev` and `u_new_dev` (ping-pong buffering), so no new memory is allocated per iteration.
4. Copies the final result back to CPU memory and returns it.

1.8.3 Performance comparison

To compare fairly against the CPU implementations, a representative floor plan (building 10000) was timed. The CPU Numba solver ran until convergence, while the CUDA solver ran for a fixed 20 000 iterations. Table 4 summarises the results.

Table 4: Solver performance comparison on a single floor plan (building 10000).

Solver	Time (s)	Iterations	Time per iteration (ms)
NumPy CPU	6.19	3,602	1.72
Numba CPU (JIT)	1.31	3,602	0.36
Numba CUDA (GPU)	1.26	20,000	0.063

The wall clock times for the Numba CPU and CUDA GPU solvers appear almost identical (1.31 s vs. 1.26 s), which is misleading. The GPU solver performs 5.5 times more iterations in roughly the same time, meaning the GPU is approximately **5.7**× faster per iteration:

$$\frac{0.36 \text{ ms}}{0.063 \text{ ms}} \approx 5.7 \times \text{faster per iteration.}$$

If the GPU solver were run for the same number of iterations as the CPU reference (3,602), the estimated time would be:

$$1.26 \text{ s} \times \frac{3,602}{20,000} \approx 0.23 \text{ s per floor plan,}$$

which corresponds to a $5.7\times$ speedup over the Numba CPU solver and a $26.9\times$ speedup over the NumPy baseline. The similar wall clock time is therefore a coincidence caused by the fixed iteration count being 5.5 times higher than what early stopping would require.

The per iteration speedup is real but modest relative to what a GPU can theoretically deliver. Two factors limit the benefit. First, the grid size of $514 \times 514 \approx 264,000$ points is small relative to the V100's 5,120 CUDA cores, so not all cores are occupied each iteration. Second, 20,000 separate kernel launches from Python accumulate non-trivial fixed overhead per launch, which reduces the effective throughput.

1.8.4 Estimated runtime for all floor plans

The mean solver time measured across 10 floor plans was 1.258 s. Extrapolating to the full dataset:

$$1.258 \text{ s} \times 4,571 \approx 5,750 \text{ s} \approx 95.8 \text{ min.}$$

This estimate assumes 20,000 fixed iterations per floor plan. Since the CPU reference converged in approximately 3,602 iterations on average, adding early stopping to the GPU solver would reduce the estimated runtime proportionally:

$$95.8 \text{ min} \times \frac{3,602}{20,000} \approx 17 \text{ min.}$$

1.9 Question 9

1.9.1 Part 1

We took 50 samples from the dataset to run the 2 solutions. The reference solution had a run time of 454 s, while the cupy solution took 135 s to run which is an increase in performance.

Table 5: Resource usage comparison between the Reference and CuPy solutions for $N = 50$ samples.

Metric	Reference Solution	CuPy Solution
Run Time (sec)	454.00	135.00
Turnaround Time (sec)	455.00	149.00
CPU Time (sec)	452.37	133.73
Max Memory (MB)	239.00	100.00
Average Memory (MB)	211.40	99.33
Max Processes	4	4
Max Threads	7	10

The LSF batch submission script used to configure the environment and execute the job is detailed in Listing 3.

Listing 3: LSF batch submission script used for the benchmarking runs.

```

1  #!/bin/bash
2  #BSUB -B -N
3  #BSUB -J python
4  #BSUB -W 20
5  #BSUB -n 4
6  #BSUB -R "rusage[mem=256MB]"
7  #BSUB -o python_%J.out
8  #BSUB -e python_%J.err
9  #BSUB -gpu "num=1:mode=exclusive_process"
10 #BSUB -q c02613
11 #BSUB -R "span[hosts=1]"
12 # Initialize Python environment
13 source /dtu/projects/02613_2025/conda/conda_init.sh
14 conda activate 02613_2026
15 # Run Python script
16 python cupy_proj_new.py 50

```

1.9.2 Part 2

As there are 4571 samples in the full dataset, the estimated runtime for the reference solution is:

$$t_{\text{ref}} = \frac{4571}{50} \times 454 \text{ s} \approx 692 \text{ minutes}$$

With the CuPy solution, the estimated runtime is:

$$t_{\text{cupy}} = \frac{4571}{50} \times 135 \text{ s} \approx 206 \text{ minutes}$$

1.9.3 Part 3

The surprising part about the performance is that the speedup is only around 3.36x faster which is low considering we switched to a GPU implementation. This could mean that in the current cupy solution there are inefficiencies or bottlenecks.

1.10 Question 10

We profiled the solution with the nsys profiler. The main issue was indicated to be that there were a lot of calls between the CPU and GPU. The `GPUMemtimesum` and `GPUMemsizesum` statistics show that `CUDA memcpy DtoH` is called 1942 times while only transferring a total of 0.007 MB of data. Additionally, `cuLaunchKernel` is called 10217 times which further hints at the fact that there might be an unnecessarily large amount of calls being made, thus the GPU kernel is being launched repeatedly which creates overhead.

We fixed the solution where we reworked the Jacobi matrix function to use cupy operations instead of boolean comparisons that require coordination with the CPU. The calculations were further optimized by using a fuse-kernel approach with the `@cp.fuse()` decorator. We made the function accept an entire 3d array so that the floorplans are not given one-by-one. This initially ran into memory allocation issues when processing the whole dataset, so we adapted it to process them in batches.

1.11 Question 11

For the optional performance improvement, the fastest CPU solution from Questions 6 and 7 was scaled further using an LSF job array. The full list of floorplans was split across 4 array jobs. Each job processed one shard of the dataset, and each shard still used the optimized `numba + dynamic` implementation internally with 8 worker processes and chunk size 1. After all array jobs finished, the shard CSV files were merged into one final result file.

This gives two levels of parallelism: multiprocessing inside each job and job-level parallelism across the LSF array. The merged result contained 4571 rows and 4571 unique building IDs, so every floorplan was processed exactly once. The merged summary reported a mean solver time of 4.77 seconds per building and a total of 39746490 Jacobi iterations.

The four array shards took 539.54, 613.71, 651.11, and 963.34 seconds, respectively. Since the full job-array runtime is limited by the slowest shard, the effective total runtime was about 963 seconds, or 16 minutes. This is substantially faster than the approximate 38 minute estimate for running the optimized CPU implementation in a single job, although it is slower than the ideal 4-way split because the shards were not equally difficult.

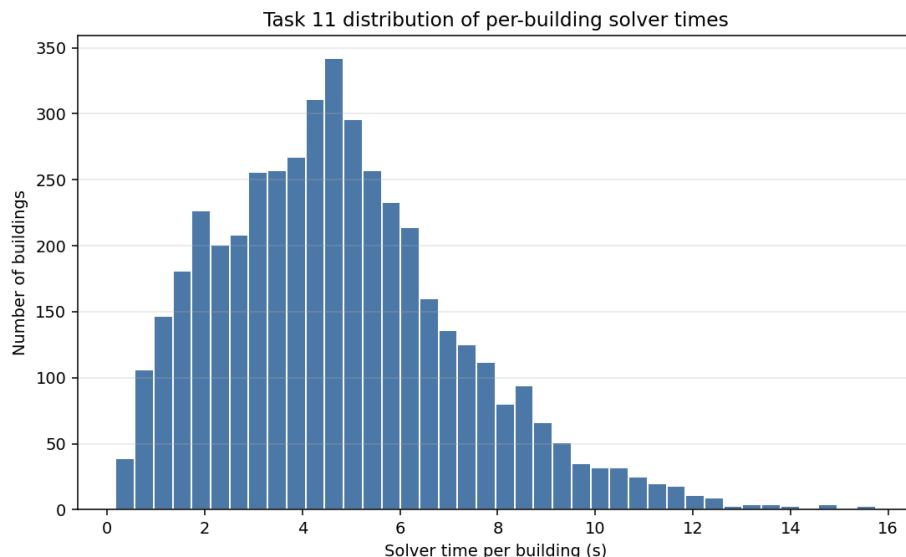


Figure 4: Distribution of per-building solver times from the merged Question 11 results. The spread in runtimes explains why both dynamic scheduling and balanced job-array partitioning matter for this workload.

The job-array approach therefore improved the total runtime further, but the result also shows that load balancing remains important at the job level. A more balanced partitioning of floorplans would likely reduce the runtime further. Figure 4 shows that per-building solver times vary substantially, with a tail of slower floorplans. This explains why equally sized shards can still have unequal runtimes.

1.12 Question 12

To process the whole dataset we used the improved cupy solution from Question 10. With increasing the batch size to 200 the solution took 1099 s to run (less than 19 minutes).

We got the following results:

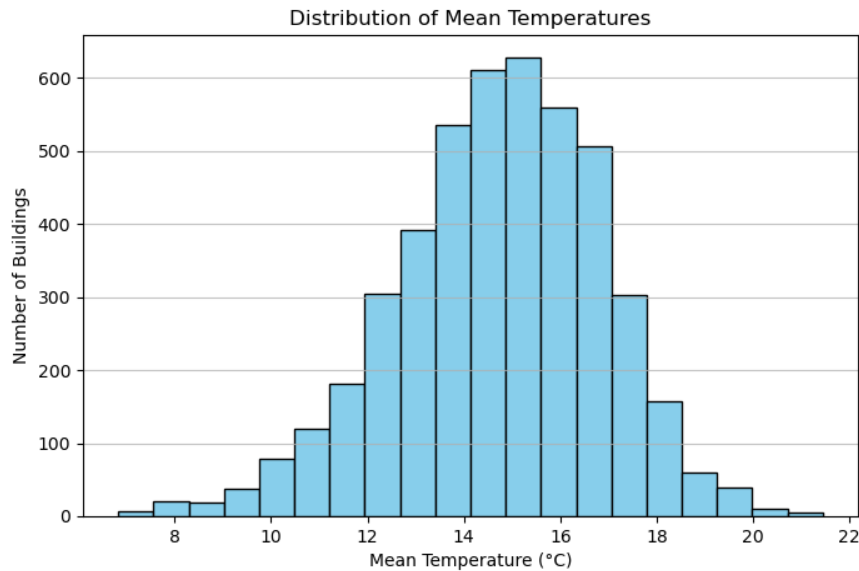


Figure 5: Distribution of mean temperatures

Table 6: Temperature Analysis

Metric	Value
Average Mean Temperature	14.71 °C
Average Temperature Standard Deviation	6.80 °C
Buildings with $\geq 50\%$ of area above 18 °C	813
Buildings with $\geq 50\%$ of area below 15 °C	2,462

List of Figures

1	A comparison between two spanning subfigures.	1
2	Speedup comparison: Measured vs. Amdahl's Law.	3
3	Measured speed-up for the CPU implementations on a subset of 50 floorplans. The difference between <code>numpy + static</code> and <code>numpy + dynamic</code> shows the effect of dynamic scheduling.	4
4	Distribution of per-building solver times from the merged Question 11 results. The spread in runtimes explains why both dynamic scheduling and balanced job-array partitioning matter for this workload.	10
5	Distribution of mean temperatures	11

List of Tables

1	Performance profile summary of the Jacobi solver.	2
2	Measured Runtime and Speed-up ($N = 50$)	3
3	Comparison between static and dynamic scheduling for the NumPy CPU im- plementation.	4
4	Solver performance comparison on a single floor plan (building 10000). . . .	7
5	Resource usage comparison between the Reference and CuPy solutions for $N = 50$ samples.	8
6	Temperature Analysis	11